

3. HAFTA: ALGORİTMA VE İŞ AKIŞ TASARIMI RAPORU

Ders Sorumlusu: Öğr. Gör. Muhammet Esat Özdağ

Teslim Tarihi: 06/10/2026

1. ÖĞRENCİ VE PROJE BİLGİLERİ

Adı Soyadı: Gencer Yıldız

Öğrenci No: 254309026

Proje Adı: Dijital Sinema Biletleme ve Yönetim Sistemi

2. ALGORİTMA TASARIMI VE ÇEKİRDEK İŞ MANTIĞI

(Bu bölümde, projenizin arka planında çalışacak temel süreçlerin (örneğin: web sayfasından veri çekme, veriyi filtreleme, formatlama) algoritmalarını adım adım veya sözde kod (pseudo-code) mantığıyla açıklayınız.)

DuckCine – Arka Plan Algoritmaları (Kısa Özet)

Süreç 1: Veri Çekme (Canlı Koltuk Haritası)

- **Amaç:**** Seçilen seansın koltuk durumunu sayfa yenilenmeden güncel tutmak.
- **Girdi → Çıktı:**** Seans no → koltuk listesi (boş / dolu / seçili / pasif).
- **Adımlar:**** Tarayıcı 4 sn'de bir istek atar → sunucu dolu ve seçili koltukları bulur → her koltuğa durum verir → harita yeniden çizilir, kaybedilen koltuk varsa uyarılır.
- HER 4 SANİYEDE (sekme görünürse)
- cevap ← fetch GET /api/seanslar/{id}/koltuklar/
- HER koltuk İÇİN
- durum ← pasif | secili (benim) | dolu (satıldı / rezerve) | bos
- HaritayıÇiz(cevap); seçtiğim koltuk dolduysa → uyar
- **Yapı:**** Tarayıcıda asenkron, sunucuda senkron.

Süreç 2: Veri Temizleme / Ayrıştırma (Gelen Form Verisi)

1. **Amaç:** Gelen veriyi güvenli ve tutarlı hale getirmek.
2. **Girdi → Çıktı:** Ham JSON → temiz veri veya alan bazlı hata listesi.
3. **Adımlar:** JSON çözülür → eksik alanlar toplanır → e-posta küçük harfe çevrilir, boşluklar kırpılır → biçim, tekillik ve şifre kuralları kontrol edilir → şifre hash'lenir.
4. $veri \leftarrow JSON.\text{çöz}(gövde)$ // bozüksa HATA
5. EĞER zorunlu alan boş İSE HATA eksikAlanlar
6. $eposta \leftarrow küçükHarf(kırp(eposta))$
7. EĞER eposta geçersiz / kayıtlı VEYA şifre zayıf İSE HATA
8. $şifre \leftarrow HASH(şifre)$
9. **Yapı:** Sunucuda senkron. (Gemma cevabı da benzer şekilde kırpılıp "düşünme" kısmından arındırılır.)

Süreç 3: Veri Filtreleme (Arama ve Film Önerisi)

1. **Amaç:** Filmleri aramaya göre süzmek ve 4 soruya göre en uygun 5 filmi önermek.
2. **Girdi → Çıktı:** Arama metni / mod, kitle, süre, zaman → film listesi / puana göre sıralı öneriler.
3. **Adımlar:** Uygun seansı olan vizyondaki filmler alınır → aile seçildiyse +7 yaş filmler elenir → puanlanır → sıralanıp ilk 5 film döner.
4. HER film f İÇİN (vizyonda VE seçilen zamanda seansı var)
5. $puan \leftarrow (mod \text{ eşleşirse } 3) + (kitle \text{ eşleşirse } 2) + (süre \text{ uygunsa } 1)$
6. EĞER $puan \leq 1$ İSE atla
7. $toplam \leftarrow puan + f.puan / 10$
8. DÖNDÜR toplam'a göre ilk 5
9. **Yapı:** Arama 200 ms bekleyip asenkron gider; filtreleme sunucuda tek sorguyla yapılır.

Süreç 4: Koltuk Rezervasyonu (Eşzamanlılık Kontrolü)

1. **Amaç:** Aynı koltuğun iki kişiye satılmasını engellemek, koltukları 10 dk ayırmak.
2. **Girdi → Çıktı:** Seans + en fazla 10 koltuk → sipariş no ya da "koltuk dolu" hatası.
3. **Adımlar:** Seans kilitlenir → süresi dolanlar temizlenir → koltuklar boş mu bakılır → sipariş ve rezerve biletler oluşturulur → kilit kalkar.
4. İŞLEM BAŞLAT
5. $seans \leftarrow KİLİTLE(seansId)$ // SELECT ... FOR UPDATE
6. EĞER seçilen koltuklardan biri dolu İSE HATA "Az önce alındı"
7. $sipariş \leftarrow oluştur(bitiş = şimdi + 10 dk)$
8. her koltuk için REZERVE bilet oluştur // tekillik kuralı = 2. güvence
9. İŞLEMİ ONAYLA
10. **Yapı:** Sunucuda senkron; istekler sırayla işlenir.

Süreç 5: Ödeme ve Çıktı Üretme (Formatlama)

1. ****Amaç:**** Ödemeyi almak, bileti kesinleştirmek, fatura, bildirim ve e-posta üretmek.
2. ****Girdi → Çıktı:**** Sipariş no + kart + sözleşme onayı → satılmış bilet, PDF fatura, e-posta.
3. ****Adımlar:**** Sipariş kilitlenir → süre kontrol edilir → POS'tan cevap beklenir → başarılıysa bilet "satıldı" olur, değilse koltuklar bırakılır → PDF fatura, bildirim ve e-posta üretilir.
4. siparis ← KİLİTLE(siparisNo)
5. EĞER süre dolmuş İSE koltukları bırak; HATA
6. sonuc ← POS.odemeAl(tutar, kart)
7. EĞER başarısız İSE koltukları bırak; HATA
8. biletler ← SATILDI // kart no ve CVV saklanmaz
9. PDF fatura → bildirim → e-posta // hata satışı geri almaz
10. ****Yapı:**** Sunucuda senkron zincir; kullanıcı sonucu tüm adımlar bitince görür.

Süreç 6: Süresi Dolan Rezervasyonları Temizleme

1. ****Amaç:**** 10 dk içinde ödenmeyen koltukları tekrar satışa açmak.
2. ****Girdi → Çıktı:**** (İsteğe bağlı) seans no → serbest bırakılan bilet / sipariş sayısı.
3. ****Adımlar:**** Süresi geçmiş rezerve biletler ve bekleyen siparişler bulunur → tek toplu güncellemeyle "süresi doldu" yapılır.
4. biletler ← REZERVE VE bitiş ≤ şimdi
5. siparişler ← BEKLEMEDE VE bitiş ≤ şimdi
6. TOPLU_GÜNCELLE(durum ← SÜRESİ_DOLDU)
7. ****Yapı:**** Senkron. Her rezervasyon isteğinde otomatik çalışır; `rezervasyon\_temizle` komutuyla elle de çalıştırılabilir.

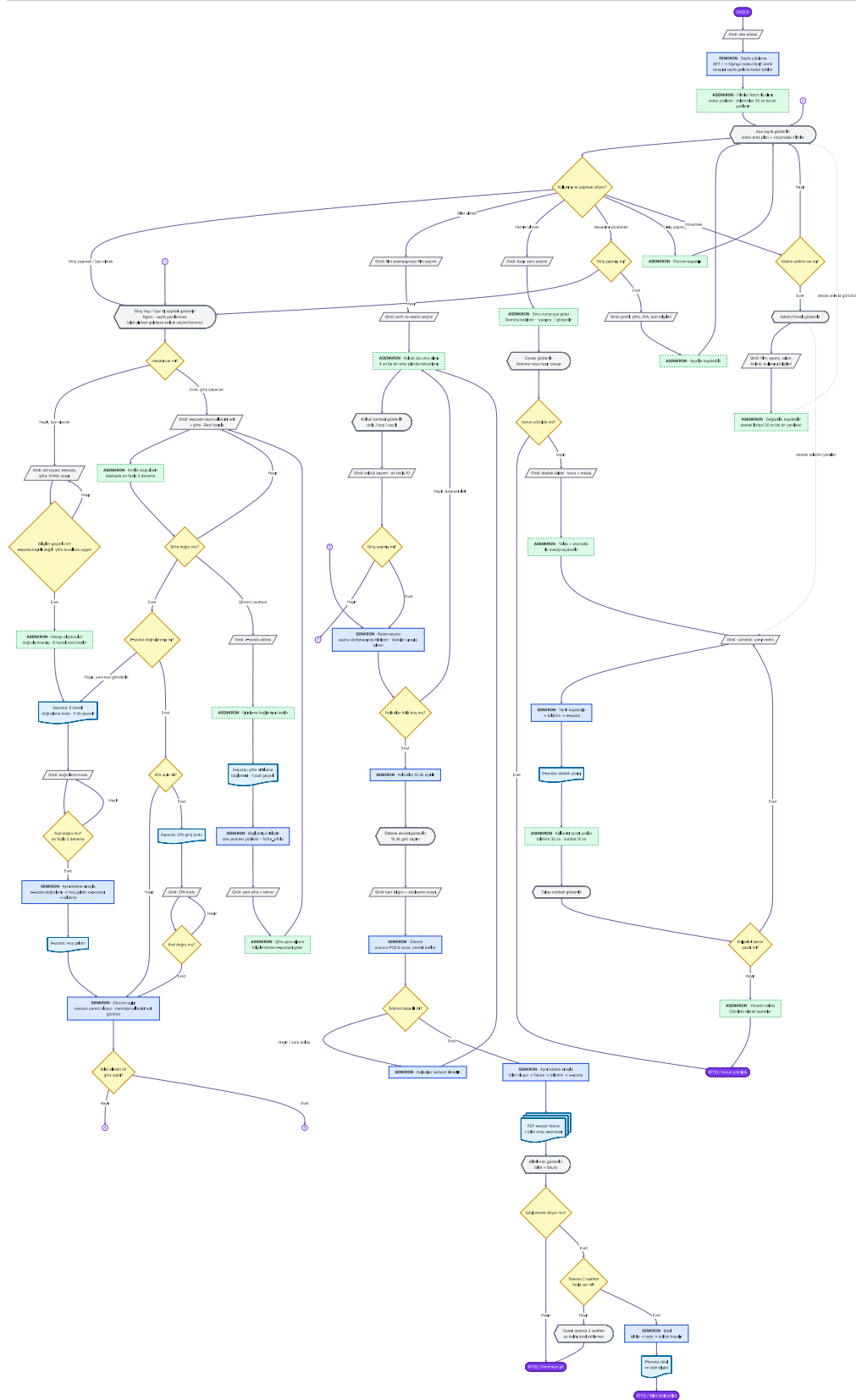
Süreç 7: Doğrulama Kodu ve Deneme Sınırı (Güvenlik)

1. ****Amaç:**** E-posta / 2FA kodlarını güvenli kontrol etmek ve kaba kuvvet denemelerini engellemek.
2. ****Girdi → Çıktı:**** Kod + IP → doğru / yanlış ya da 429 "Çok fazla deneme".
3. ****Adımlar:**** 6 haneli kod üretilir, hash'lenip 5 dk geçerli saklanır → girilen kod süre, deneme sayısı (en fazla 5) ve hash ile kontrol edilir → her istek türü için IP başına sayaç tutulur.
4. EĞER kod süresi dolmuş VEYA deneme ≥ 5 İSE HATA
5. EĞER HASH(kod) ≠ kayıt İSE deneme ← deneme + 1; HATA
6. kod ← kullanıldı
7. sayac ← artır(istekTürü + IP)
8. EĞER sayac > limit İSE 429 // giriş: dakikada 5
9. ****Yapı:**** Sunucuda senkron; istek işlenmeden önce kontrol edilir.

Süreç 8: Yapay Zekâ Destek Cevabı

1. ****Amaç:**** Hazır soruya onaylı bilgiye dayalı Gemma cevabı üretmek; model yoksa hazır cevabı vermek.
2. ****Girdi → Çıktı:**** Soru no (serbest metin yok) → cevap, kaynak (gemma / sss), ilgili 3 soru.
3. ****Adımlar:**** Soru ve onaylı cevabı (gerekirse kullanıcının biletleleri) modele gönderilir → cevap temizlenir → hata, zaman aşımı veya boş cevapta hazır cevaba geçilir.
4. $sss \leftarrow SSS.bul(soruId)$
5. $DENE\ cevap \leftarrow Temizle(Gemma.sor(sss.soru + sss.onayliCevap))$ // en fazla 90 sn
6. $HATA\ OLURSA\ cevap \leftarrow sss.cevap$
7. DÖNDÜR cevap, kaynak, ilgili 3 soru
8. ****Yapı:**** Tarayıcıda asenkron ("yazıyor..." göstergesi); sunucu modeli bekler.

3. İŞ AKIŞ DİYAGRAMI (FLOWCHART)



4. SÜRÜM KONTROL (GITHUB/GITLAB) GÜNCELLEMELERİ

(Bu hafta deponuza yüklediğiniz dokümanları veya taslak kodları belirtiniz.)

- Depo (Repository) URL: <https://github.com/GencerYILDIZ/-Dijital-Sinema-Biletleme-ve-Yonetim-Sistemi>
- Bu Hafta Yapılan Commit'ler (Yüklemeler):
3.Hafta_Rapor_Gencer_YILDIZ_254309026.pdf

5. HAFTALIK DEĞERLENDİRME VE KARŞILAŞILAN ZORLUKLAR

(İş akışlarını ve algoritmaları tasarlarken laboratuvarında karşılaştığınız mantıksal veya teknik zorlukları ve bunları nasıl aştığınızı açıklayınız.)

- Karşılaşılan Zorluklar: Projenin mimarisini ve algoritmalarını iş akış şemasına dökme sürecini hızlandırmak için yapay zekâdan yararlandım. Ancak üretilen ilk şemalar standart akış şeması kurallarına tam uymuyordu. Kullanıcıdan veri alınan adımlarda girdi sembolü (paralelkenar) kullanılmamış, bitiş sembolü (oval) "Sinemaya git" gibi bitiş belirtmeyen bir ifadeyle etiketlenmiş ve siteye ilk kez gelen kullanıcının üye olma ve giriş yapma akışı şemada yer almamıştı. Ayrıca şema büyüdükçe birbirini kesen uzun oklar okunabilirliği düşürüyordu.
- Çözüm Yaklaşımları: Akış şeması sembollerinin anlamlarını bildiğim için çıktıları kurallara göre kontrol ettim ve hataları tek tek belirterek düzeltme istedim:

1. Veri girişi olan tüm adımların paralelkenar ile gösterilmesini sağladım.
2. Bitiş noktalarının "BİTİŞ / Sinemaya git" biçiminde, hem bitiş hem sonucu gösterecek şekilde yazılmasını istedim.
3. Üye olma, e-posta doğrulama, giriş, 2FA ve şifre sıfırlama adımlarını şemaya ekletip senkron ve asenkron işlemlerin ayrı renklerle işaretlenmesini istedim.
4. Karmaşıklaşan okların yerine standart bağlantı noktası sembolünün kullanılmasını sağladım.

Son olarak bütün şemayı ve algoritma açıklamalarını kendim okuyup projenin gerçek işleyişiyle karşılaştırdım; doğruluğundan emin olduktan sonra rapora ekledim. Şemanın draw.io biçimini de alarak ileride kolayca düzenlenebilir hale getirdim.

- Gelecek Haftanın Hedefi (Veri Tabanı Tasarımı): Gelecek hafta Django ORM kullanarak PostgreSQL üzerinde veri tabanı şemasını oluşturmayı hedefliyorum. Kullanıcı, film, salon, koltuk, seans, sipariş, bilet ve fatura tabloları ile aralarındaki ilişkiler tanımlanacak, ER diyagramı çizilecek ve migration'larla tablolar oluşturulacak.

Öğrenci İmzası:

Gmy.